

Mezcla de fertilizantes

Una empresa agrícola fabrica dos tipos de fertilizante mezclando tres compuestos químicos (A, B, C). Tiene disponibles 120 kg de A, 80 kg de B y 150 kg de C por semana. Cada tonelada producida genera una ganancia de \$180 (Fertilizante F1) y \$220 (Fertilizante F2).

	Compuesto A (kg)	Compuesto B (kg)	Compuesto C (kg)	Ganancia (\$/ton)
Fertilizante F1 (x_1)	3	2	5	180
Fertilizante F2 (x_2)	4	1	3	220
Disponible (kg)	120	80	150	—

- Define las variables de decisión.
- Formula la función objetivo.
- Plantea todas las restricciones del problema incluyendo no negatividad.
- Clasifica el modelo como lineal/no lineal y continuo/discreto. Justifica

a) x_1 = Toneladas de fertilizante f_1 producidas
x Semana

x_2 = Toneladas de fertilizante f_2 producidas
x Semana

b) función Objetivo

$$\text{Max } Z = 180x_1 + 220x_2$$

c) Restricciones:

$$A \bullet 3x_1 + 4x_2 \leq 120 \text{ kg (disponible)}$$

$$B. 2x_1 + x_2 \leq 80 \text{ kg}$$

$$C. 5x_1 + 3x_2 \leq 150 \text{ kg}$$

$$x_1 \geq 0, x_2 \geq 0$$

D) Clasificación del modelo

- Es lineal pq no tiene potencias, raíces ni multiplicación entre variables
- y continua pq solo toma valores enteros

```
from pulp import *

# Crear el problema
prob = LpProblem("Fertilizantes", LpMaximize)

# Para no negatividad
x1 = LpVariable("F1", lowBound=0) # toneladas de F1
x2 = LpVariable("F2", lowBound=0) # toneladas de F2

# (maximizar ganancia)
prob += 180*x1 + 220*x2

# Restricciones
prob += 3*x1 + 4*x2 <= 120 # Compuesto A
prob += 2*x1 + 1*x2 <= 80  # Compuesto B
prob += 5*x1 + 3*x2 <= 150 # Compuesto C

# Resuelve solito
prob.solve(PULP_CBC_CMD(msg=0))

# Resultados
print("F1 =", round(x1.varValue,1), "toneladas")
print("F2 =", round(x2.varValue,1), "toneladas")
print("Ganancia máxima = $", round(value(prob.objective),1))
```

- ... F1 = 21.8 toneladas
- F2 = 13.6 toneladas
- Ganancia máxima = \$ 6927.3

Producción semanal de una fábrica

Una fábrica produce sillas y mesas. La producción de la semana siguiente depende de la demanda de la semana actual según las siguientes relaciones:

$$S_{n+1} = 0.6 \cdot S_n + 0.2 \cdot M_n + 40$$

$$M_{n+1} = 0.1 \cdot S_n + 0.5 \cdot M_n + 20$$

Donde S_n es el número de sillas producidas en la semana n y M_n el número de mesas. Se parte de $S_0 = 200$ sillas y $M_0 = 80$ mesas.

- Identifica las variables dependientes e independientes.
- Clasifica el modelo y justifica por qué es lineal y discreto.
- Calcula la producción de las semanas 1 y 2.
- ¿Hacia qué valores de equilibrio tienden S_n y M_n si la tendencia continúa? Plantea el sistema para encontrarlo (no es necesario resolverlo).

a) Variables dependientes e independientes

N = Independiente, prácticamente controla el sistema

S_n y M_n = Dependiente del número de semana (n)

b) Clasificación del modelo

Lineal: porque no tiene potencias, raíces ni multiplicaciones entre variables

Discreto: P_q n no puede tomar valores decimales, o sea no existen 1,33 semanas, es 1 o 2...

c) Producción en semana 1 y 2

$$S_{(1)+1} = 0,6(200) + 0,2(80) + 40$$

$$= 120 + 16 + 40$$

$$S_{(1)+1} = 176$$

$$M_{(1)+1} = 0,1(200) + 0,5(80) + 20$$

$$= 20 + 40 + 20$$

$$M_{(1)+1} = 80$$

$$S_{(2)+1} = 0,6(176) + 0,2(80) + 40$$

$$= 105,6 + 16 + 40$$

$$S_{(2)+1} = 161,6$$

$$M_{(2)+1} = 0,1(176) + 0,5(80) + 20$$

$$= 17,6 + 40 + 20 = 77,6$$

d) Punto de equilibrio

O sea un
$$\begin{cases} S_{n+1} = S_n = S^* \\ M_{n+1} = M_n = M^* \end{cases}$$

Introducimos en el sistema

$$S^* = 0,6 S^* + 0,2 M^* + 40$$

Sillas

$$\begin{aligned} S^* - 0,6 S^* &= 0,2 M^* + 40 \\ &= 0,4 S^* - 0,2 M^* = 40 \end{aligned}$$

$$M^* = 0,1 S^* + 0,5 M^* + 20$$

$$\begin{aligned} 0,5 M^* &= 0,1 S^* + 20 \\ -0,1 S^* + 0,5 M^* &= 20 \end{aligned}$$

Mesas

```
import numpy as np
S = 200
M = 80

S1 = 0.6*S + 0.2*M + 40
M1 = 0.1*S + 0.5*M + 20

S2 = 0.6*S1 + 0.2*M1 + 40
M2 = 0.1*S1 + 0.5*M1 + 20

print("Semana 1:", S1, "sillas |", M1, "mesas")
print("Semana 2:", S2, "sillas |", M2, "mesas")
```

```
... Semana 1: 176.0 sillas | 80.0 mesas
Semana 2: 161.6 sillas | 77.6 mesas
```

```
import numpy as np
#Mirar en los apuntes de donde sale el sistema
#0.4S-0.2M,=40
#0.5M-0.1S=20
```

```
A=np.array([[0.4, -0.2],
            [-0.1, 0.5]])
```

```
B=np.array([40,20])
```

```
Res=np.linalg.solve(A,B)
```

```
print("S*=",Res[0])
```

```
print("M*=",Res[1])
```

```
... S*= 133.33333333333334
    M*= 66.66666666666667
```

Epidemia en una ciudad

En una ciudad de 10,000 habitantes se modela la propagación de una enfermedad. El modelo SIS divide la población en: $S(t)$ = susceptibles e $I(t)$ = infectados. La tasa de nuevos contagios es proporcional al producto $S \cdot I$ (contacto entre grupos) y los infectados se recuperan a tasa $\gamma = 0.1$ por día. La tasa de contagio es $\beta = 0.00003$ por día.

Como la población total es constante: $S(t) + I(t) = 10,000$, el modelo se reduce a:

$$\frac{dI}{dt} = \beta \cdot (10,000 - I) \cdot I - \gamma \cdot I$$

Se inicia con $I(0) = 50$ personas infectadas.

a) Identifica las variables y parámetros del modelo.

b) Explica por qué este modelo es no lineal. Señala el término responsable.

c) Escribe la condición inicial.

d) Encuentra los puntos de equilibrio igualando $dI/dt = 0$ (sin resolver la EDO).

e) Interpreta físicamente cada punto de equilibrio encontrado.

Variables y parámetros...

T = tiempo en días, independiente

$I(t)$ = Infectados x día, dependiente

$S(t)$ = Susceptibles = $10.000 - I(t)$

Parámetros

Tasa $d?$

β 0,00003 Tasa de contagio diario

γ 0,1 Tasa de recuperación diaria

N 10.000 Población total

b) P_g no es lineal?

$$\frac{dI}{dt} = \beta \cdot (10.000 - I) \cdot I - \gamma \cdot I$$

$$= 10.000\beta I - \beta I^2$$

I^2 ya no es lineal

c) $I(0) = 50$ infectados

d) Cuando deja de cambiar

$$\frac{dI}{dt} = 0$$

$$\begin{aligned} \therefore \beta \cdot (10.000 - I)I - \gamma I &= 0 \\ &= I (\beta \cdot (10.000 - I) - \gamma) = 0 \end{aligned}$$

1er Caso: $I = 0$

2do Caso: $\beta (10.000 - I) - \gamma = 0$

$$0,00003 (10.000 - I) = 0,1$$

$$10.000 - I = \frac{0,1}{0,00003} = 3.333,33$$

$$10.000 - 3.333,33$$

$$= 6.666,67$$

e) Si hay 0 infectados, no hay enfermedad, o se estabiliza a los 6666,67 infectados



```
from scipy.integrate import odeint #Entendi que es para las ecuaciones d  
import numpy as np
```

```
# Parámetros
```

```
beta = 0.00003
```

```
gamma = 0.1
```

```
N = 10000
```

```
I0 = 50
```

```
# Ecuación diferencial
```

```
def modelo(I, t):
```

```
    return beta * (N - I) * I - gamma * I
```

```
# Tiempo
```

```
t = np.linspace(0, 200, 1000) #mas puntos mas preciso
```

```
# Resolver
```

```
I = odeint(modelo, I0, t)
```

```
print("Infectados al día 200:", round(I[-1][0], 2))
```

```
... Infectados al día 200: 6666.67
```

Modelo logístico de pesca

Una laguna tiene una población de peces que crece logísticamente. Cada año se extraen $H = 80$ peces por pesca controlada. La capacidad máxima de la laguna es $K = 1000$ peces y la tasa de crecimiento intrínseca es $r = 0.4$.

$$P_{n+1} = P_n + r \cdot P_n \left(1 - \frac{P_n}{K}\right) - H$$

Se parte de una población inicial de $P_0 = 400$ peces.

- Identifica variables y parámetros.
- Explica por qué el modelo es no lineal y discreto.
- Calcula P_1 y P_2 (población al final del año 1 y 2).

Optimización

- Encuentra el o los puntos de equilibrio del sistema (cuando $P_{n+1} = P_n$). Plantea y resuelve la ecuación resultante.
- Interpreta físicamente los equilibrios obtenidos: ¿qué le ocurre a la población si inicia por debajo del equilibrio inestable?

a) Variables:

n = El año

P_n = Población de peces en el año n

Parámetros:

$r = 0.4$ ^{Valor} ^{d?} Tasa de crecimiento intrínseca

$K = 1000$ Capacidad máxima de la laguna

$H = 80$ Peces extraídos x año

b) Por qué es NO lineal y discreto

$$P_{n+1} = P_n + r \cdot P_n \left(1 - \frac{P_n}{K}\right) - H$$

$$= r \cdot P_n - \frac{P_n^2}{K}$$

P_n^2 ya no es lineal

c)

```
#4c)
r = 0.4
K = 1000
H = 80
P = 400 #Inicial

for n in range(1, 3):
    Pnuevo = P + r*P*(1-P/K)-H
    P = Pnuevo
    print(f"P{n} =", round(P, 1))
```

```
P1 = 416.0
P2 = 433.2
```

d)

```
import numpy as np
coef=[0.0004, -0.4, 80]
raiz = np.roots(coef)

print("P1* =", round(raiz[0], 1))
print("P2* =", round(raiz[1], 1))
```

```
... P1* = 723.6
    P2* = 276.4
```

e) Se debe tener cuidado, si es menor se extingue
y si es mayor se estabilizan en 724 machos